

Technical Interview Brief

Adaptive Navigation and Conflict-Aware Path Planning (Logistics Focus) ROS 2 Humble · Gazebo Classic 11 · nav2 · colcon/ament · 9 packages · 256 tests

How to use this: §1 is the opening answer. §2–7 are the walkthrough. §8 is the deep dive you will be pushed into. §9 is the numbers you must not fumble. §10 is the strongest material you have — the bugs. §11 is question drill.

1. The 60-second answer

"It's a two-robot cooperative navigation stack on ROS 2 Humble and nav2. The robots are deliberately heterogeneous — a heavy 120 kg mapper and a light 30 kg scout — and the whole design turns on one rule: **every physical constant is declared once in `robot_models.yaml` and every consumer derives from it.** Nav2's controller limits, the velocity smoother, the Gazebo plugin ceilings and the safety envelope are all computed from that one file at launch time. On top of stock nav2 I wrote five things nav2 doesn't provide: a BSP sensor validation layer that everything downstream consumes instead of raw topics; a log-odds map fusion node that merges both robots' SLAM into one grid; a selective-mapping policy that throttles updates in well-travelled areas but never throttles frontier; two custom costmap plugins — a slope layer that prices ramps and a fleet-trajectory layer that writes peers' predicted positions as time-decayed cost; and a jerk-limited motion smoother feeding a safety override node that is the sole publisher of `cmd_vel`. The world is generated rather than drawn — one Python model emits the SDF, the elevation map, the waypoints and the landmark manifest, so the geometry the planner prices and the geometry Gazebo simulates cannot diverge."

2. Assignment → implementation map

Brief	Where it lives
§1 Heterogeneous fleet	<code>robot_models.yaml</code> , <code>amr_core::ModelLibrary</code>
§1 Multi-level world with ramps	<code>amr_gazebo/world_builder.py</code> (generated)
§2.1 Cooperative SLAM + fusion	<code>amr_mapping::MapFusion</code> , <code>map_fusion_node</code>
§2.2/2.3 Selective mapping	<code>amr_mapping::SelectiveMappingPolicy</code>
§2.4 Concurrent goals	<code>send_goals.py</code>
§2.5/2.6 Ramp cost, last resort	<code>amr_navigation::SlopeCostModel</code> + <code>SlopeLayer</code>
§3.1–3.3 Motion smoothing	<code>amr_fleet_control::MotionSmoother</code>
§3.4 Peer trajectories	<code>TrajectoryPredictor</code> → <code>FleetTrajectoryLayer</code>
§3.5 Yielding protocol	<code>ConflictDetector</code> + <code>YieldPolicy</code> in <code>traffic_control_node</code>
§3.6–3.8 Safety override	<code>SafetyMonitor</code> + <code>safety_override_node</code>
§4.1 BSP validation	<code>amr_sensor_bsp</code> (Lidar/Imu/Camera validators)
§4.2 Scalability	namespacing + <code>fleet_ten_robots.yaml</code>
§5.1 README/build	<code>README.md</code> , <code>docs/RUNBOOK.md</code>
§5.2 Refactoring	<code>docs/REFACTORING.md</code>

3. Package architecture

Nine packages, layered so the dependency graph is acyclic and the core is testable without ROS.

```

amr_core      (no ROS deps – pure C++: model library, fleet config, geometry)
  +-- amr_sensor_bsp      (sensor validation)
  +-- amr_mapping         (map fusion, selective mapping)
  +-- amr_navigation      (nav2 costmap plugins)
  +-- amr_fleet_control   (smoother, traffic, safety, broadcaster)
amr_msgs      (interfaces)
amr_description (xacro – one parameterised URDF for both robots)
amr_gazebo     (world generation)
amr_bringup    (launch, config, demos, consistency checks)

```

Why amr_core has no ROS dependency: it makes the model library and fleet config unit-testable in isolation, with no node lifecycle, no executor and no discovery. Three C++ test files run against it directly.

Key types in amr_core

Type	Purpose
LidarSpec, ImuSpec, CameraSpec	Per-sensor physical spec
DynamicLimits	Velocity/accel/jerk envelope + derating curves
SafetySpec	$k, d_{\min}$ for $d_{\text{safe}} = k \cdot v^2 + d_{\min}$
RobotProfile	One model — all of the above plus mass/geometry
RobotInstance	One <i>robot</i> — a profile plus name, spawn pose, priority
FleetPolicy	Horizon, conflict margin, yield timings
FleetConfig	Roster + policy + resolved model library
ModelLibrary	Loads and validates robot_models.yaml

Interview point — the distinction between RobotProfile and RobotInstance. A profile is a *model* (heavy_mapper); an instance is a *deployed robot* (amr1, at (−18.5, 2.2), priority 100). Ten robots can share two profiles. That distinction is what makes fleet_ten_robots.yaml a config change.

4. Complete node inventory

Per robot (namespaced /amr1, /amr2)

Node	Package	Subscribes	Publishes
bsp_validation	amr_sensor_bsp	scan_raw, imu_raw, camera_raw/image_raw	scan, imu, camera/image_raw, sensor_health
slam_toolbox	(stock)	scan	/map → remapped to map, TF <ns>/map→<ns>/odom
selective_mapping	amr_mapping	map (own SLAM grid)	map_contribution, map_update_stats
controller_server	nav2	scan, costmaps	cmd_vel_nav
planner_server	nav2	global costmap	path
smoother_server, behavior_server, bt_navigator, waypoint_follower, lifecycle_manager	nav2	—	—
velocity_smoother	amr_fleet_control	cmd_vel_nav, traffic_directive	cmd_vel_smoothed
safety_override	amr_fleet_control	cmd_vel_smoothed, scan, odom	cmd_vel, safety_status
trajectory_broadcaster	amr_fleet_control	odom, plan	/fleet/trajectories
robot_state_publisher	(stock)	joint_states	TF

Fleet singletons (root namespace)

Node	Subscribes	Publishes
map_fusion	/amr*/map_contribution	/map, static TF map→<ns>/map
traffic_control	/fleet/trajectories	/amr*/traffic_directive
dynamic_obstacle_driver	—	obstacle poses to Gazebo

Custom message types

PredictedTrajectory (robot_id, footprint_radius, yield_priority, TrajectoryPoint[]), TrajectoryPoint (pose, time_from_start, speed), FleetTrajectories, TrafficDirective (action PROCEED/SLOW/YIELD/HOLD, speed_scale, conflicting_robot, time_to_conflict, reason), SafetyStatus (halt_active, min_obstacle_distance, safe_distance, current_speed, reaction_latency_us, reason enum), SensorHealth (status OK/DEGRADED/INVALID, counters, measured_rate_hz, active_faults), MapUpdateStats, PayloadState. Services: SetPayload, SetSafetyOverride.

5. TF tree

```
map                                ← fleet root
+-- amr1/map                       ← STATIC, published by map_fusion
|   +-- amr1/odom                  ← slam_toolbox
|       +-- amr1/base_footprint    ← Gazebo diff-drive
|           +-- amr1/base_link
|               +-- amr1/lidar_link, imu_link, camera_link → camera_optical_link
|               +-- amr1/cargo_deck, lidar_mast
|               +-- amr1/{left,right}_wheel_link, {front,rear}_caster_link
+-- amr2/map → ... (identical)
```

Why each robot has a private map frame. Each runs its own SLAM, which means each has its own drifting estimate. Anchoring them into a shared map via a static transform derived from the known dock pose gives the fleet a single root without forcing the two SLAM instances to agree instantaneously.

Interview trap: "Why not one SLAM for both?" — because slam_toolbox is single-robot; sharing one instance would mean one robot's scan matching corrupting the other's pose. Fusion at the *grid* level, not the *solver* level.

6. Launch sequence, step by step

```
fleet.launch.py
+- simulation.launch.py           → gzserver, gzclient, dynamic_obstacle_driver
+- fleet_control.launch.py       → traffic_control, map_fusion (root namespace)
+- for each robot, staggered by spawn_delay * index:
    robot.launch.py (PushRosNamespace <robot>)
    +- spawn_entity.py           (Gazebo model from xacro)
    +- robot_state_publisher
    +- bsp_validation_node
    +- velocity_smoother_node
    +- safety_override_node
    +- trajectory_broadcaster_node
    +- after_startup_delay:
        +- slam.launch.py        → slam_toolbox, selective_mapping_node
        +- navigation.launch.py  → nav2 stack + lifecycle_manager
```

Why robots are staggered. Spawning several models into Gazebo simultaneously reliably wedges the spawn service, and bringing up N nav2 stacks at once trips the lifecycle bonds. spawn_delay defaults to 6 s.

Why SLAM and nav2 are delayed by startup_delay. They need the Gazebo spawn to have produced odometry TF first; otherwise the costmaps come up with no transform and time out.

The two-stage parameter substitution (a favourite question)

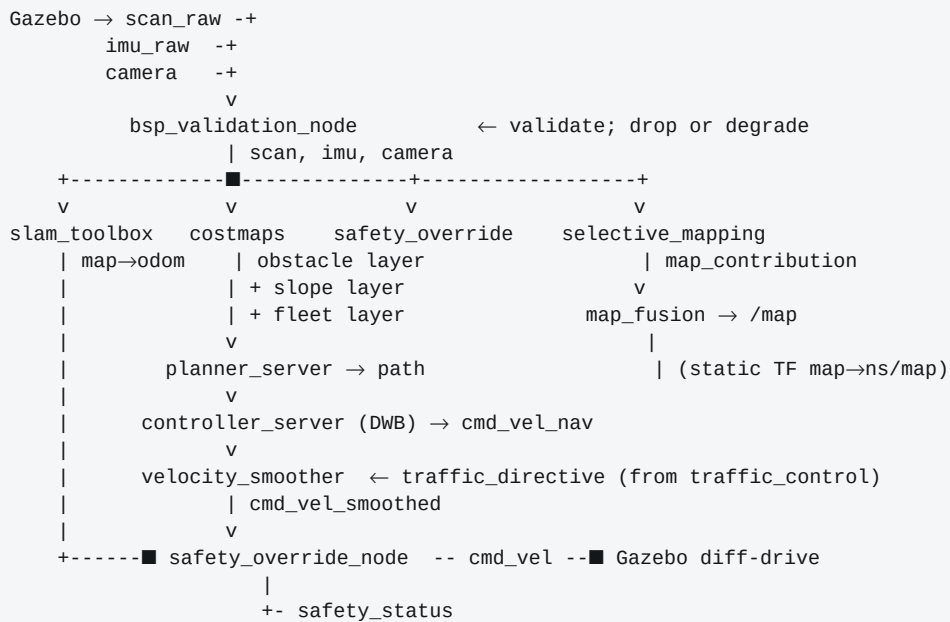
nav2_params.yaml is a **template** processed in two passes:

1. **Textual pass** — `<ns>`, `<map_width>`, `<map_origin_x>` etc. are string substituted in `navigation.launch.py`.
2. **Key-based pass** — `nav2_common.launch.RewrittenYaml` rewrites parameters by key name from `fleet_loader.nav2_substitutions()`.

Why both? `RewrittenYaml` matches on **key name** and replaces every occurrence in the file. That is correct for `robot_radius` and catastrophically wrong for `global_frame`, which must be `map` in the global costmap and `<ns>/odom` in the rolling local costmap. Same for width: rewriting it would give the 8×8 m rolling local costmap the width of the whole 48 m warehouse. So anything whose value differs *per occurrence* goes through the textual pass, and a guard test fails if a frame key ever reappears in the key-based map.

There is also an assertion that no `<...>` placeholder survives expansion — a missed placeholder would reach `nav2` as an unparseable value.

7. One control cycle, end to end



The three arbitration layers, in order of authority:

1. `traffic_control` **scales** the target (cooperative, jerk-limited).
2. `velocity_smoother` **shapes** it (acceleration + jerk bounded).
3. `safety_override` **replaces** it (hard, immediate, non-negotiable).

They are deliberately different mechanisms: a yield is a *controlled* stop that the smoother shapes; a safety halt bypasses the smoother entirely.

8. Subsystem deep dives

8.1 BSP sensor validation (`amr_sensor_bsp`)

Not a HAL — a validation layer. The node subscribes to raw Gazebo topics and republishes validated data. **No navigation node subscribes to a raw topic**, so unvalidated data cannot reach a planner even by accident.

SensorValidator base class → LidarValidator, ImuValidator, CameraValidator. Each returns a ValidationResult with `ShouldForward()` and maintains ValidationStatistics.

Faults are graded, not binary:

Check	Severity	Rationale
empty scan	hard	nothing to consume
wrong beam count	hard	driver misconfigured; angles would be wrong

Check	Severity	Rationale
all returns non-finite	hard	sensor not producing data
geometry disagrees with model (geometry_tolerance 0.02 rad)	hard	angles cannot be trusted
some out-of-bounds returns	soft	individual bad beams — clamp and carry on
stale timestamp (staleness_seconds 0.5)	soft	reported; the safety monitor runs its own watchdog

The **explicit brief requirement** is `ImuValidator`: angular velocity beyond `imu.max_angular_velocity` (2.5 rad/s heavy, 4.5 scout) logs a warning naming the robot, the measured rate, the limit and the model. Deliberately **soft** — a 40 rad/s reading on a warehouse AMR is far more likely a bad sample than a real event, and halting on it would be the wrong response.

`CameraValidator` checks mean intensity against `[min_mean_intensity, max_mean_intensity] = [6, 249]` to catch a black or saturated frame — both of which produce a perfectly valid `sensor_msgs/Image` at the right rate.

Ground-return rejection (the clever bit). When the chassis pitches nose-down onto a ramp, the LiDAR's forward beams strike the floor and arrive as obstacles. The validator uses the *validated IMU pitch* to predict the floor range and delete those returns:

```
tilt = sin(pitch) * cos(bearing)      ← PER BEAM, not one value
range = lidar_height / tilt
```

The `cos(bearing)` term is essential: a beam pointing sideways on a pitched chassis has zero effective tilt and never meets the floor. An earlier version applied one forward range to all 1080 beams and carved fan-shaped voids out of the scan. Suppression is also *banded* (`±ground_rejection_tolerance 25%`) rather than "everything beyond", and limited to a $\pm 60^\circ$ forward arc.

8.2 Map fusion (`amr_mapping::MapFusion`)

Merges per-robot occupancy grids into one warehouse-wide map.

Log-odds accumulator, not per-cell maximum. Taking the maximum means one robot's transient false positive — a pedestrian, a scan artefact — permanently marks a cell the other robot can see is free. Log-odds lets disagreement resolve.

Parameter	Value	Why
occupied_delta	+0.65	per-observation weight, deliberately modest
free_delta	−0.45	asymmetric: seeing-through is weaker evidence than hitting
log_odds_min/max	±3.5	clamp — without it a long-held belief becomes unfalsifiable and the map stops tracking reality
free_threshold	0.25	render as free below this probability
occupied_threshold	0.65	render as occupied above

Also publishes the **static transforms** `map → <robot>/map` from each robot's known dock pose. That is what gives the fleet a single TF root.

8.3 Selective mapping (`SelectiveMappingPolicy`) — brief §2.2/2.3

Three bands per cell:

Band	Condition	Update period
Frontier	within <code>frontier_radius</code> (1.0 m) of unknown space	always — never throttled
Explored	visited < <code>saturation_visits</code> (8)	<code>explored_period</code> 1.0 s
Saturated	visited ≥ 8	<code>saturated_period</code> 5.0 s

The key design decision: the visit count comes from **where the robot has actually driven**, not from where it has looked. A corridor traversed twenty times is saturated; a shelf face observed from twenty angles is not. That directly encodes "reduce the update frequency for areas it has repeatedly traversed" from the brief.

On a fresh map almost everything is frontier and nearly all updates flow; on a mature map suppression exceeds 90%. MapUpdateStats publishes suppression_ratio and policy_state so it is measurable, not asserted.

8.4 Slope layer (amr_navigation) — brief §2.5/2.6

SlopeCostModel samples the generated elevation map, computes the local gradient by finite difference, and prices it:

```

θ ≤ free_angle (2°)           → FREE
θ ≥ max_traversable_angle (16°) → LETHAL
otherwise:
  f = (θ - free) / (max - free)
  cost = base_cost + (max_cost - base_cost) · f^γ

```

with base_cost = 200, max_cost = 252, γ = 1.0.

The 253 trap — the single best detail in this project. nav2 defines INSCRIBED_INFLATED_OBSTACLE = 253 and LETHAL_OBSTACLE = 254. nav2_smac_planner's Node2D::isNodeValid **refuses** any cell at or above

253. So 252 is expensive and 253 is impassable — there is no gradual approach to "never". A ramp priced 253 is not avoided, it is *unusable*, which silently breaks the requirement that a ramp is taken when it is the only way through.

The model's own clamp targeted kLethal - 1 = 253, which is off by one; the ceiling is 252 and a test asserts it.

Why base_cost is high and the curve linear. A steep ramp prices itself out anyway. The 5° service ramp is the one that looks like a shortcut, and a squared curve is exactly what made it cheap. With the base high, the routing question is no longer *how steep* a cell is but *whether it is a ramp at all*.

Verified against the world by Dijkstra using Smac's own pricing (cell × (1 + cost_travel_multiplier × cost/252), multiplier 3.0):

```

dock_a → east_staging, doorway open   : 36.10   ramp used: no
dock_a → east_staging, doorway sealed : 54.15   ramp used: yes
dock_a → mezzanine_storage            : 46.03   ramp used: yes (reachable)

```

18 m of extra distance accepted rather than climb. Both halves asserted, because either alone is trivially satisfiable.

SlopeLayer is the nav2 CostmapLayer plugin wrapper. It **bakes** the cost grid once at activation (the field is static) and applies it over the whole costmap extent rather than a rolling window — a rolling update would leave stale slope cost behind the robot.

8.5 Fleet trajectory layer — brief §3.4

trajectory_broadcaster publishes a **4 s projection at 10 Hz** on /fleet/trajectories, sampled every 0.2 s, built from the robot's odometry and its current nav2 plan. Note the asymmetry: the *conflict detector* uses the full 4 s (policy.horizon_seconds), while the costmap layer consumes only the first **3 s** (fleet_layer.horizon_seconds). Arbitration should look further ahead than the cost field, so a conflict is negotiated before it is ever painted as cost.

FleetTrajectoryLayer (a nav2 costmap plugin) subscribes and writes cost into each robot's **local** costmap for its *peers*:

	Cost
peer's present position	present_cost 200
far end of the 3 s layer horizon	horizon_cost 30
inflation around each point	inflation_margin 0.15 m

The design point: a peer's predicted position is not an obstacle, it is a *probability that decays with lookahead*, and the cost should say so. A flat lethal cost over the whole horizon would make the robots refuse to share a corridor at all.

trajectory_timeout 1.0 s — a stale prediction is worse than none, because it looks authoritative.

8.6 Traffic control and yielding — brief §3.5

ConflictDetector — space-time, not space. For each pair it walks both predicted paths in lockstep and finds the first sample where

```
centre_distance < r_a + r_b + conflict_margin (0.35 m)
```

returning a Conflict{robot_a, robot_b, time_to_conflict, separation, required_separation, x, y}. Paths older than trajectory_timeout are ignored.

YieldPolicy — priority-based, from the roster (heavy_mapper 100, light_scout 50). The lighter, lower-priority AMR-2 yields to the mission-critical AMR-1, exactly as the brief specifies.

Policy parameter	Value	Meaning
horizon_seconds	4.0	prediction lookahead
sample_period	0.2	trajectory sample spacing
conflict_margin	0.35	extra separation beyond the two radii
conflict_react_seconds	3.0	time-to-conflict at which SLOW engages
hard_yield_seconds	1.5	time-to-conflict at which YIELD engages
slow_speed_scale	0.35	target velocity multiplier when slowing
max_yield_seconds	12.0	anti-starvation: boost priority after this
yield_cooldown_seconds	4.0	prevents oscillation after release
traffic_rate_hz	10.0	arbitration rate

Two details worth volunteering:

1. **The directive scales the target velocity; it does not command a stop.** The motion smoother then shapes the deceleration, so a yield is a jerk-limited slow-down rather than a cut. That is what makes it a "temporary, controlled stop" in the brief's words.
2. **Robots with no conflict receive an explicit PROCEED every cycle.** Publishing a positive "you are clear" means a robot can treat *silence* as a fault and fail safe, instead of assuming it may continue.

Anti-starvation: EffectivePriority() boosts a robot that has been yielding continuously for max_yield_seconds, so a low-priority robot in a busy corridor cannot be blocked indefinitely.

8.7 Motion smoother — brief §3.1–3.3

MotionSmoother::SmoothAxis is the most mathematically involved piece. Five steps per axis per tick:

1. **Clamp the target, not the output.** Clamping after the acceleration has been chosen truncates it without warning, and that step in acceleration is an unbounded jerk spike — occurring exactly when the robot reaches top speed.
2. **Choose the acceleration bound.** Braking (acceleration opposing current velocity) may use decel_limit, which is larger than accel_limit. Physically true and the safer asymmetry.
3. **Terminal-approach law — three ceilings, tightest wins:**
 - (a) magnitude_limit — the actuator envelope;
 - (b) the largest acceleration that can still be walked back to zero at the jerk limit before the remaining velocity error is consumed;
 - (c) $|e| / dt$ — never overshoot within this tick.

Ceiling (b) is what turns a trapezoidal ramp into an S-curve. **The derivation matters and is a strong interview answer:**

The continuous-time form is $\sqrt{(2 \cdot j \cdot |e|)}$. That is *not safe in a discrete controller*, because the acceleration is chosen from the error measured at the **start** of the tick, so the ramp-down runs one full tick behind. Unwinding an acceleration a therefore consumes $a^2/(2j) + a \cdot dt$ of velocity, not $a^2/(2j)$.

Solving $a^2/(2j) + a \cdot dt = |e|$ for a gives the bound actually used:

```
a_cap = sqrt( (jdt)^2 + 2j|e| ) - jdt
```


A sweep over both robot models, jerk limits 0.6–4.0 and tick rates 10–50 Hz shows this lands exactly on target with zero overshoot and peak jerk at (never above) the limit. The half-step form $a^2/(2j) + a \cdot dt/2$ still overshoots by ~0.5 mm/s; the continuous form by ~30 mm/s.

4. Jerk envelope applied last, so it can only *tighten* step 3, never loosen it. Note what is deliberately absent: no re-clamp to `magnitude_limit` afterwards. The envelope itself moves — it shrinks with speed and payload and switches bounds at zero velocity — and forcing the acceleration inside a newly shrunk envelope in one tick is itself a jerk discontinuity. Instead it walks back inside at the jerk limit.

5. Feed forward the *achieved* acceleration, not the requested one. That is what keeps the jerk limit honest after a clamp.

Dynamic state (brief §3.1: "based on its dynamic state"). `DynamicLimits` derates multiplicatively and independently:

```
effective = nominal · (1 - payload_derating·load_ratio) · (1 - speed_derating·speed_ratio)
```

with `payload_derating` 0.3 and `speed_derating` 0.2. A fully loaded robot at top speed is the most constrained state — the physically sensible ordering. Payload is injected at runtime via the `SetPayload` service.

Why a separate node rather than nav2 parameters: `nav2`'s controllers have no concept of jerk. That is precisely why the smoother sits downstream of the controller.

Fails open. If the smoother stops receiving `cmd_vel_nav` it ramps to a controlled stop. A smoother that fails *closed* could freeze a robot with nothing in front of it.

8.8 Safety override — brief §3.6–3.8

`safety_override_node` is the sole publisher of `cmd_vel`. Everything upstream publishes a *proposed* velocity on `cmd_vel_smoothed`. That single-writer arrangement is what makes the override real rather than advisory — there is no path by which an upstream command reaches the base while the monitor objects.

Envelope: $d_{\text{safe}}(v) = k \cdot v^2 + d_{\text{min}}$

	k	d_min	d_safe at v_max
heavy_mapper	0.85	0.45 m	0.93 m
light_scout	0.42	0.30 m	1.12 m

Quadratic because braking distance goes as $v^2/2a$. `check_model_consistency.py` verifies $k \cdot v_{\text{max}}^2 + d_{\text{min}} \leq \text{lidar.range_max}$ for every model — an envelope the sensor cannot see to is decorative.

Design details worth volunteering:

- **The decision runs inside the scan callback**, not on a timer, so reaction latency is the sensor period rather than sensor period + scheduler quantum. `SafetyStatus.reaction_latency_us` records scan acquisition stamp → halt publication.
- **Fails closed.** No scan within `sensor_timeout_seconds` (0.35 s) halts the robot. Contrast the smoother, which fails open.
- **Hysteresis + minimum hold.** Release requires `d_safe` + hysteresis *and* `min_hold_seconds` (0.4 s) elapsed, so the robot cannot chatter in and out of halt at the loop rate.
- **Guarded cone ±70°**, not 360° — an obstacle directly behind a forward-moving robot is not a reason to stop.
- `halt_blocks_rotation` (default true) — when false, a halt zeroes linear velocity but permits rotation, so the robot can turn away.
- Manual engagement via the `SetSafetyOverride` service.

8.9 World generation (`amr_gazebo/world_builder.py`)

One Python model emits **five** artefacts: the SDF world, a PGM/YAML elevation map, `waypoints.yaml`, `dynamic_obstacles.yaml`, and `maps/warehouse_landmarks.txt`.

Why generated rather than drawn: a hand-drawn world and a hand-written elevation map are two representations of the same geometry that diverge silently — and the slope layer would then price ramps that are not where it thinks they

are.

Current world: 44 × 30 m, 34 static bodies, 4 ramps (5.0°, 6.0°, 6.0°, 9.0°), 2 decks (hump 0.38 m, mezzanine 0.40 m), 5 dynamic obstacles (4 animated pedestrian actors + 1 rigid box), 9 named goals.

Scenario features designed to exercise specific requirements:

Feature	Exercises
The Pinch — 2.10 m firewall doorway	Yielding protocol. Sized so two robots abreast fall inside the 1.27 m conflict threshold, forcing the protocol rather than letting them squeeze past.
Hump Bridge — 0.38 m deck, two 6° ramps	The sloped alternative to the flat doorway. Wider and more comfortable <i>except</i> for gradient, so a planner ignoring slope will use it.
Mezzanine — 0.40 m deck, 5° and 9° ramps	Reachable only by ramp → forces the "only viable path" case.
Rack blocks, 4.0 m aisles, 3.0 m cross-aisles	Real aisle-routing for the global planner.
Animated pedestrians with per-bone collision spheres	Genuine dynamic obstacle avoidance.

Pedestrians are Gazebo <actor> elements so they animate (run.dae), but actors carry **no collision geometry by default** — the AMRs drove straight through them. Explicit per-bone collision spheres (feet, legs, up-legs, hips, lower back) form a continuous column from floor to chest so a 2D scan plane at any height intersects something.

9. Numbers you must know cold

Robot models

	AMR-1 heavy_mapper	AMR-2 light_scout
Payload capacity	120 kg	30 kg
Chassis mass	85 kg	28 kg
Chassis L×W×H	0.90 × 0.62 × 0.34 m	0.58 × 0.44 × 0.26 m
Footprint radius	0.55 m	0.37 m
max_vel_x	0.75 m/s	1.4 m/s
max_accel_x	0.35 m/s ²	1.10 m/s ²
max_decel_x	0.70 m/s ²	1.60 m/s ²
max_vel_theta	0.9 rad/s	1.9 rad/s
max_accel_theta	0.80 rad/s ²	2.40 rad/s ²
max_jerk_x	0.6 m/s ³	2.5 m/s ³
LiDAR	1080 beams, 25 m, 15 Hz, h 0.60 m	720 beams, 16 m, 30 Hz, h 0.48 m
IMU	100 Hz, ω limit 2.5 rad/s	200 Hz, ω limit 4.5 rad/s
Safety k / d_min	0.85 / 0.45 m	0.42 / 0.30 m
Yield priority	100	50

Costmaps

	Global	Local
Size	48 × 34 m, static	8 × 8 m, rolling
Frame	map	<ns>/odom
Update / publish	1.0 / 1.0 Hz	10.0 / 5.0 Hz
Layers	static, slope , obstacle, inflation	obstacle, fleet , inflation
inflation_radius	0.60	0.60
cost_scaling_factor	3.0	3.0

Planner SmacPlanner2D, cost_travel_multiplier 3.0, allow_unknown true, tolerance 0.25 m. Controller DWB at 20 Hz, sim_time 1.7 s, vx_samples 20, vtheta_samples 40, PathAlign/PathDist.scale 32, BaseObstacle.scale 0.02.

The inflation arithmetic (comes up constantly)

A robot needs $2 \times \text{inflation_radius} + \text{robot_width}$ of clear space before any pose in a corridor costs nothing.

With inflation_radius 0.60 and a 1.10 m robot: **2.30 m minimum**. That single line explains the 4.0 m aisles, the 3.0 m cross-aisles and the 2.10 m doorway.

10. The bug catalogue — your strongest material

Method used throughout: **turn the symptom into a number and compare it against a physical or configuration quantity**. In every case below the error message pointed somewhere other than the cause.

#	Symptom	Number that explained it	Root cause
1	Starting point in lethal space!	SLAM mapped a 19×13 cell box = 0.95×0.65 m — the chassis footprint	LiDAR mounted <i>inside</i> the chassis; every beam hit its own body. Fixed by raising it onto a collision-free mast.
2	Robot is out of bounds of the costmap!	Robot reported at ($-37.0, 4.4$); spawn $x = -18.5$, and $-18.5 - 18.5 = -37$	Pose anchored twice — odometry_source was WORLD <i>and</i> map_fusion anchored again. Set to ENCODER.
3	Same message, earlier cause	Global costmap defaulted to 5×5 m	map_extents never specified. Added <code><map_*></code> textual placeholders.
4	Same message, third cause	—	slam_toolbox publishes on absolute /map; the remap ('map', 'map') matched nothing. An unmatched remap rule is not an error.
5	Robot stalls on every ramp	Plant ceiling $1.40 \text{ rad/s}^2 \times 0.115 \text{ m} = \mathbf{0.16 \text{ m/s}^2}$ vs 1.03 m/s^2 of gravity on a 6° slope	<code><max_wheel_acceleration></code> is angular ; it was set from a linear value with no /wheel_radius.
6	Robot wedges between racks	$2.40 \text{ m aisle} - 2 \times 0.75 \text{ m inflation} = \mathbf{0.90 \text{ m}}$ usable lane, for a 1.10 m robot	Aisles too narrow for the inflation. Widened to 4.0 m.
7	Scan develops fan-shaped voids	$\sin(\text{pitch}) \cdot \cos(\text{bearing}) = \mathbf{0}$ sideways	One forward ground-return range applied to all 1080 beams. Made per-beam and banded.
8	Robots hunt and stall in the doorway	$2.00 \text{ m} - 2 \times 0.75 \text{ m} = \mathbf{0.50 \text{ m}}$ cost-free band; controller had to hold centreline to $\pm 25 \text{ cm}$ <i>while turning in</i>	The one passage the inflation arithmetic was never applied to. Widened to 2.10 m, inflation padding cut $0.20 \rightarrow 0.05 \text{ m}$.
9	AMRs drive through pedestrians	—	Gazebo <code><actor></code> has no collision geometry. Added per-bone spheres forming a continuous column.
10	AttributeError: can't set attribute 'clients'	—	<code>rcipy.node.Node.clients</code> is a read-only property. Renamed; added an AST guard test.

#	Symptom	Number that explained it	Root cause
11	min_laser_range silently default 0.0	—	Parameter was spelled minimum_laser_range, which slam_toolbox ignores without error .
12	Five C++ tests failing silently	Ramp gradients hard-coded as 8.7°/14.8°; world had been regenerated to 6.0°/9.0°	colcon test would have failed on submission. Fixed structurally: world_builder now emits a landmark manifest the tests read.

The meta-lesson, and a good thing to say out loud: on two occasions a guard test *passed its own negative control* — meaning it was decoration, not a guard. One compared the mapper's scan smear only *relative* to the scout's, which was satisfiable by making the mapper crawl; another set ratio floors *below* the ratios the fleet already shipped with. Since then every fix is verified by reverting it and confirming the test fails.

11. Question drill

"Why not use AMCL?" The brief requires the robots to map an *initially unknown* warehouse. AMCL localises against a known map. slam_toolbox in mapping mode builds it, and map fusion merges both robots' contributions.

"Why is the safety node the only cmd_vel publisher?" Because an override that shares the topic is advisory, not authoritative. With a single writer there is no path by which an upstream command reaches the base while the monitor objects. Everything upstream publishes a *proposal*.

"Why is the smoother a separate node?" nav2's controllers bound acceleration but have no concept of jerk. The brief asks for jerk limiting based on dynamic state, so it has to live downstream.

"How would you scale to 50 robots?" Roster is config, so launch scales already. The bottleneck is traffic_control: ConflictDetector is $O(n^2)$ pairwise. At 50 robots that is 1225 pairs at 10 Hz. Fix is spatial hashing — only test pairs whose bounding circles overlap — which is a contained change because detection is already isolated behind ConflictDetector::Detect.

"What's the weakest part?" Map fusion anchors each robot's SLAM frame from its *known dock pose* and never corrects it. If a robot's SLAM drifts, the merged map degrades and there is no inter-robot loop closure to pull it back. The correct answer is a pose-graph optimiser over shared landmarks, which is substantial work.

"Why generate the world?" Because the slope layer prices geometry from an elevation map while Gazebo simulates geometry from an SDF. If those are authored separately they diverge silently and the planner prices ramps that aren't where it thinks they are.

"How do you know the ramp requirement is met?" A Dijkstra over the generated elevation map using Smac's own cell pricing shows the planner accepts 18 m of extra distance rather than climb, *and* uses the bridge when the doorway is sealed. Both halves are asserted — either alone is trivially satisfiable.

"What does cost_travel_multiplier do?" Smac prices a cell as $\text{cell_size} \times (1 + \text{cost_travel_multiplier} \times \text{cost}/252)$. It is the exchange rate between costmap cost and path length. At 3.0, a cell at cost 214 is 3.5× as long as a free one.

"Difference between a yield and a safety halt?" Different mechanisms on purpose. A yield is *cooperative and predictive* — the traffic controller scales the target velocity and the smoother shapes a jerk-limited slow-down. A safety halt is *reactive and unconditional* — it bypasses the smoother and replaces the command outright.

12. Known limitations — state these before you are asked

- 1. Descending a ramp can stall the robot.** The BSP writes inf into suppressed ground beams; nav2's ObstacleLayer defaults inf_is_valid: false, which *discards* them — they neither mark nor clear. Phantom obstacles marked during the pitch transition can never be raytrace-cleared. The robot is stopped by an obstacle its own ground-rejection created and then blinded itself to. One-line fix (inf_is_valid: true) implemented and

verified, but not in the submitted build.

2. **AMR-1 cannot see AMR-2.** AMR-1's scan plane is 0.600 m; AMR-2's tallest *colliding* surface is 0.384 m → the beam passes 216 mm over it. Masked throughout because the fleet trajectory layer shares poses over the network, so they avoid each other in the costmap while being mutually invisible to the sensor the safety override watches.
3. **Wheel slip on slopes → odometry drift.** `max_wheel_torque` allows 783 N at the contact patch against ~637 N of grip, when climbing needs 71 N. With ENCODER odometry every slipping revolution integrates as distance travelled.
4. **Casters bridge the ramp foot.** Caster contact points are coplanar with the drive wheels, so at the concave vertex the fore and aft casters bridge it and lift the driven wheels ~2.6 cm. A four-wheel skid-steer conversion removes this but introduces lateral scrub that diff-drive odometry cannot observe — the correct answer is EKF fusion of wheel odometry with the IMU gyro (`robot_localization`), not attempted.
5. Camera BSP validator implemented and unit-tested but never exercised at runtime. Screenshare video not recorded.

13. Build and demo commands

```
colcon build --symlink-install && source install/setup.bash

ros2 launch amr_bringup fleet.launch.py          # full stack
ros2 run  amr_bringup send_goals.py \
  --goal amr1=heavy_storage --goal amr2=packing_bay_4 # concurrent goals

ros2 run amr_bringup demo_slope_planning.py      # A/B/C incl. slope-off control
ros2 run amr_bringup demo_conflict.py            # yielding protocol
ros2 run amr_bringup demo_safety_override.py     # override vs hostile cmd stream
ros2 run amr_bringup check_model_consistency.py
colcon test && colcon test-result --verbose
```

Scale test: `ros2 launch amr_bringup fleet.launch.py fleet_config:=$(ros2 pkg prefix amr_bringup)/share/amr_bringup/config/fleet_ten_robots.yaml`